ĐẠI HỌC BÁCH KHOA HÀ NỘI

TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



[Mã học phần – Tên học phần]

WikiPulse

Phân tích xu hướng Wikipedia và ưu tiên xem xét chỉnh sửa đáng ngờ theo thời gian thực

(Báo cáo kỹ thuật – bản cập nhật liên tục)

Giảng viên hướng dẫn:

[Học hàm, học vị, họ tên]

Nhóm sinh viên thực hiện:

[Họ tên] (Trọng)	[MSSV]
[Họ tên] (Thành)	[MSSV]
[Họ tên] (Linh)	[MSSV]
Hoàng Văn Nhân	20235542
[Họ tên] (Mạnh)	[MSSV]

Hà Nội - Việt Nam

2026

Quy ước cập nhật báo cáo

Báo cáo này được cả nhóm viết dần trong quá trình phát triển, để thành viên mới đọc hiểu được project và để báo cáo cuối kỳ không phải soạn lại từ đầu. Quy ước:

- Phần chưa làm chỉ có đề mục, kèm dòng *(Dự kiến, chưa thực hiện.)* – macro `\chualam`.
- Phần đã viết nhưng còn sửa theo các tuần sau có ghi chú nhỏ bên dưới – macro `\dukiensua{nội dung, tuần}`.
- Thông tin còn thiếu đánh dấu màu đỏ – macro `\canbosung`. Trên trang bìa, mỗi thành viên tự điền họ tên và MSSV của mình; trưởng nhóm điền học phần và giảng viên.
- Mỗi việc trong `docs/checkpoints.md` khi chuyển sang DONE thì owner cập nhật mục tương ứng ở đây trong cùng PR, kèm số liệu và bằng chứng tái hiện được. Không ghi kết quả chưa đo.
- Nguồn sự thật cho giao diện giữa các module vẫn là `docs/contracts.md`; báo cáo giải thích và dẫn lại, không định nghĩa contract mới.

Bảng 1: Lịch sử cập nhật báo cáo

Ngày	**Người**	**Nội dung**
15/10/2026	Nhân	Khởi tạo khung báo cáo; viết phần CP0 (kiến trúc, luồng dữ liệu, serving, kiểm thử)
15/10/2026	Nhân	Thêm use case (biểu đồ, đặc tả), biểu đồ hoạt động; viết lại Chương 2 kèm hình minh họa; chuyển tên file, lệnh sang Phụ lục A

Mục lục

Quy ước cập nhật báo cáo **i**

Danh mục thuật ngữ và viết tắt **i**

1 Giới thiệu **1**

1.1 Đặt vấn đề 1

1.2 Mục tiêu 1

1.3 Phạm vi 2

1.4 Tổ chức nhóm 2

1.5 Cấu trúc báo cáo 3

2 Cơ sở lý thuyết và công nghệ **4**

2.1 Bài toán xử lý dữ liệu luồng 4

2.2 Luồng sự kiện thay đổi của Wikipedia 5

2.3 Apache Kafka: bộ đệm sự kiện bền vững 6

2.4 Spark Structured Streaming 7

2.4.1 Mô hình micro-batch 7

2.4.2 Cửa sổ thời gian và watermark 7

2.4.3 Số tạm tính và số đã chốt 8

2.4.4 Loại bản trùng 8

2.5 Kiến trúc dữ liệu ba tầng 9

2.6 Apache Cassandra: phục vụ kết quả 9

2.7 FastAPI và Streamlit 10

2.8 Công nghệ sử dụng 11

3 Các công cụ và nghiên cứu liên quan **12**

3.1 Công cụ tuần tra của cộng đồng Wikipedia . . . 12

3.2 Mô hình đánh giá chỉnh sửa của Wikimedia . . . 12

3.3 Thống kê lượt xem và xu hướng trang . . . 12

3.4 Khoảng trống mà WikiPulse nhắm tới . . . 12

4 Phân tích, thiết kế và cài đặt **13**

4.1 Phân tích yêu cầu . . . 13

4.1.1 Tác nhân . . . 13

4.1.2 Biểu đồ use case . . . 14

4.1.3 Đặc tả các use case chính . . . 15

4.1.4 Yêu cầu chức năng . . . 16

4.1.5 Yêu cầu phi chức năng . . . 16

4.2 Kiến trúc tổng quan . . . 17

4.2.1 Các thành phần và người phụ trách . . . 17

4.2.2 Triển khai . . . 18

4.3 Đường đi của một sự kiện . . . 18

4.4 Biểu đồ tuần tự: người xem mở trang Overview . . . 19

4.5 Cấu hình và tách lần chạy . . . 19

4.6 Thu thập dữ liệu . . . 20

4.6.1 Kafka . . . 20

4.6.2 Phát lại dữ liệu mẫu . . . 20

4.6.3 Thu thập luồng thật . . . 20

4.7 Xử lý luồng . . . 20

4.7.1 Các truy vấn . . . 20

4.7.2 Định danh và loại trùng . . . 21

4.7.3 Đếm hoạt động . . . 21

4.7.4 Đặc trưng xu hướng . . . 21

4.7.5 Chấm điểm ưu tiên xem xét . . . 21

4.8 Lưu trữ . . . 21

4.8.1 Tầng Parquet . . . 21

4.8.2 Lược đồ Cassandra . . . 22

4.8.3 Ghi kết quả từ Spark sang Cassandra . . . 22

4.9 API . . . 23
4.10 Dashboard . . . 23

5 Kiểm thử và đánh giá **24**
5.1 Chiến lược kiểm thử . . . 24
5.2 Kết quả CP0 . . . 24
5.3 Xu hướng và ưu tiên xem xét trên fixture vàng . . . 25
5.4 Độ trễ end-to-end . . . 25
5.5 Khởi động lại, retry và dữ liệu đến muộn . . . 26
5.6 Chạy LIVE liên tục . . . 26
5.7 Tải và dung lượng . . . 26

6 Kết luận và hướng phát triển **27**
6.1 Kết quả đạt được . . . 27
6.2 Hạn chế . . . 27
6.3 Hướng phát triển . . . 27
6.3.1 Collector LIVE với nối lại và đếm lỗi (W2-THANH-01) . . . 28
6.3.2 Cửa sổ trượt, đặc trưng xu hướng và điểm rủi ro (W2-LINH-01) . . . 28
6.3.3 API xu hướng, chỉnh sửa đáng ngờ và chi tiết sự kiện (W2-NHAN-01) 28
6.3.4 Hoàn thiện bốn trang dashboard (W2-MANH-01) . . . 28
6.3.5 Profile HDFS và tích hợp ba tầng (W2-TRONG-01, W3-LINH-01) 28
6.3.6 Retry, latest window, TTL và kiểm thử dung lượng (W3-NHAN-01) 28
6.3.7 Khung Kubernetes và monitoring (W3-TRONG-01) . . . 28
6.3.8 Đóng băng, clone sạch và demo (tuần 4) . . . 28

A Bảng tra mã nguồn **29**

Tài liệu tham khảo **31**

Tóm tắt nội dung

Wikipedia nhận hàng trăm chỉnh sửa mỗi phút. Wikimedia công bố toàn bộ thay đổi này dưới dạng luồng sự kiện công khai (EventStreams `recentchange`) [8]. Luồng này đủ để biết trang nào đang được quan tâm và chỉnh sửa nào đáng được người tuần tra xem trước, nhưng dữ liệu đến liên tục, có trùng lặp, có sự kiện đến muộn và có bản ghi hỏng.

Báo cáo trình bày **WikiPulse**, một nền tảng xử lý luồng đầy đủ: Kafka nhận sự kiện, Spark Structured Streaming kiểm tra, chuẩn hóa, loại trùng và tổng hợp theo cửa sổ thời gian sự kiện, dữ liệu được lưu ba tầng Bronze/Silver/Gold dạng Parquet, kết quả phục vụ qua Cassandra, FastAPI và dashboard Streamlit. Hệ thống chạy được ở hai chế độ: LIVE nối với Wikimedia, và REPLAY phát lại dữ liệu mẫu cố định để kiểm thử tái hiện được.

Đến ngày 15/10/2026, nhóm đã hoàn thành mốc CP0: luồng REPLAY chạy thật từ Kafka tới dashboard cho nghiệp vụ đếm hoạt động theo wiki trong cửa sổ 1 phút, kết quả khớp bộ kết quả kỳ vọng độc lập và được kiểm chứng trên hai máy (macOS ARM64 và Windows x86_64). Trang xu hướng, ưu tiên chỉnh sửa đáng ngờ, chế độ LIVE và HDFS là việc của tuần 2–4, được liệt kê dưới dạng đề mục.

Ghi chú: Tóm tắt viết lại khi chốt CP4.

Danh mục thuật ngữ và viết tắt

Thuật ngữ	**Nghĩa**
Event time	Thời điểm sự kiện xảy ra trên Wikipedia (trường `meta.dt`), khác thời điểm hệ thống nhận
Cửa sổ (window)	Khoảng thời gian event time cố định để tổng hợp, ví dụ 1 phút
Watermark	Ngưỡng trễ tối đa hệ thống chờ sự kiện đến muộn (10 phút); qua ngưỡng thì cửa sổ được chốt
Bronze / Silver / Gold	Ba tầng dữ liệu: thô, đã chuẩn hóa, đã tổng hợp và chốt
Provisional / finalized	Số liệu tạm tính (có thể còn đổi) / số liệu đã chốt sau watermark
REPLAY / LIVE	Chế độ phát lại dữ liệu mẫu / chế độ nhận luồng thật từ Wikimedia
Upsert	Ghi đè theo khóa: có thì thay, chưa có thì thêm
DLQ	Dead-letter queue, topic chứa bản ghi không xử lý được
Canary	Sự kiện kiểm tra do Wikimedia phát vào luồng, không phải chỉnh sửa thật
Checkpoint	Trạng thái Spark lưu lại để chạy tiếp sau khi dừng; đồng thời là mốc nghiệm thu CP0–CP4 của nhóm

Chương 1

Giới thiệu

1.1 Đặt vấn đề

Luồng `recentchange` của Wikimedia phát mọi thay đổi trên các dự án Wikimedia: sửa trang, tạo trang mới, thay đổi nhật ký. Mỗi sự kiện có wiki, tiêu đề trang, người sửa, kích thước trước và sau, cờ bot, cờ ẩn danh. Từ luồng này có thể trả lời hai câu hỏi có ích:

1. **Đang có gì nóng?** Trang nào đang được sửa nhiều bất thường trong vài phút gần nhất.
2. **Nên xem chỉnh sửa nào trước?** Chỉnh sửa nào có dấu hiệu cần người tuần tra kiểm tra, ví dụ xóa nhiều nội dung, hoàn tác qua lại, hay sửa ẩn danh.

Khó khăn nằm ở tính chất của dữ liệu luồng: sự kiện đến không theo thứ tự, có bản trùng khi nguồn kết nối lại, có bản ghi hỏng, và kết quả phải cập nhật liên tục trong khi vẫn phải đúng khi hệ thống khởi động lại.

1.2 Mục tiêu

- Xây dựng luồng end-to-end Kafka → Spark → Parquet/Cassandra → API → dashboard, chạy được ở hai chế độ LIVE và REPLAY.
- Tổng hợp theo event time có watermark, loại trùng, cách ly bản ghi lỗi, đúng khi retry và khởi động lại.
- Phục vụ ba nghiệp vụ: hoạt động theo wiki, trang xu hướng, danh sách chỉnh sửa cần ưu tiên xem xét.

- Bảo vệ quyền riêng tư: tên người dùng và IP không đi ra khỏi tầng Bronze.

1.3 Phạm vi

- **Trong phạm vi:** MVP một node chạy bằng Docker Compose; mặc định hai wiki `enwiki`, `viwiki`, mở rộng bằng cấu hình; lưu Parquet cục bộ (tuần 1) rồi HDFS (tuần 2–3).
- **Ngoài phạm vi:** mô hình học máy, độ sẵn sàng cao, triển khai cloud, xác thực và phân quyền. Kubernetes và monitoring chỉ ở mức khung.
- **Nguyên tắc:** điểm rủi ro chỉ dùng để *ưu tiên xem xét*, không kết luận một chỉnh sửa là phá hoại. Không đưa dữ liệu giả vào đường demo.

1.4 Tổ chức nhóm

Nhóm 5 người, 4 tuần, khoảng 10–15 giờ mỗi người mỗi tuần. Tiến độ quản lý theo mốc nghiệm thu (Bảng 1.2) và bảng việc chi tiết trong `docs/checkpoints.md`; một việc chỉ chuyển DONE khi có bằng chứng kiểm thử.

Bảng 1.1: Phân công theo module

Thành viên	**Phụ trách**
Trọng	Core, cấu hình, Docker Compose, Makefile, tích hợp và nghiệm thu chung
Thành	Ingestion: Kafka topics, replay, collector LIVE
Linh	Xử lý Spark: validation, chuẩn hóa, loại trùng, cửa sổ, đặc trưng xu hướng và rủi ro
Nhân	Lưu trữ phục vụ Cassandra, sink từ Spark, API FastAPI
Mạnh	Dashboard Streamlit, kiểm thử chấp nhận (acceptance)

Bảng 1.2: Các mốc nghiệm thu

Mốc	**Điều kiện DONE**	**Trạng thái**
CP0	REPLAY thật qua Kafka/Spark/3 tầng Parquet/Cassandra/API/Overview	DONE (11/10/2026)
CP1	Người khác chạy lại được CP0; mỗi người nhận module, contract và test đầu tiên	Đang làm
CP2	LIVE/REPLAY, cửa sổ, rủi ro, xu hướng đúng fixture; UI thật; HDFS khởi động	Chưa
CP3	E2E trên HDFS; late data, khởi động lại, retry, Gold finalized được kiểm chứng	Chưa
CP4	Clone sạch trên máy thứ hai; replay pass; LIVE $\geq$ 15 phút; 4 trang; đủ tài liệu	Chưa

Ghi chú: Cập nhật cột trạng thái mỗi khi đạt mốc.

1.5 Cấu trúc báo cáo

- Chương 2 trình bày kiến thức nền về xử lý luồng và các công nghệ sử dụng.
- Chương 3 điểm lại các công cụ và nghiên cứu liên quan.
- Chương 4 mô tả phân tích, thiết kế và cài đặt.
- Chương 5 trình bày kiểm thử và kết quả.
- Chương 6 kết luận và liệt kê phần còn lại.

Chương 2

Cơ sở lý thuyết và công nghệ

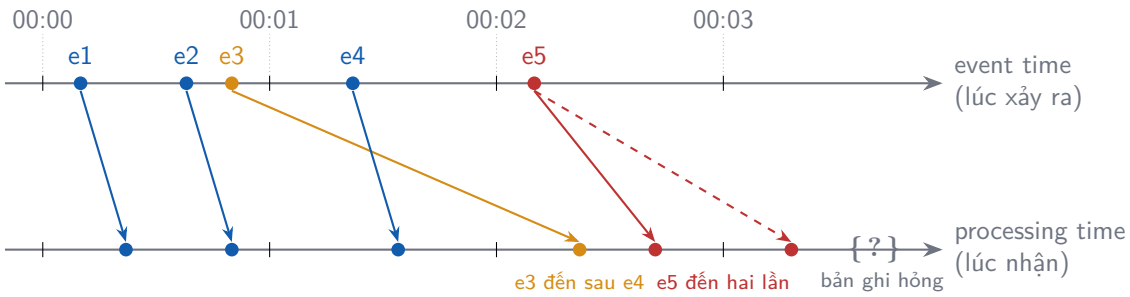
Chương này giải thích các khái niệm mà thiết kế ở Chương 4 dựa vào: dữ liệu luồng khác dữ liệu tĩnh ở đâu, Kafka giữ sự kiện thế nào, Spark tổng hợp theo thời gian ra sao khi sự kiện đến lộn xộn, vì sao dữ liệu được lưu thành ba tầng, và vì sao Cassandra được chọn để phục vụ kết quả.

2.1 Bài toán xử lý dữ liệu luồng

Dữ liệu tĩnh (một file, một bảng) có điểm bắt đầu và kết thúc: chạy truy vấn một lần là có kết quả. Dữ liệu luồng thì **không bao giờ kết thúc**: sự kiện đến liên tục, kết quả phải được tính dần và cập nhật mãi. Ngoài ra, mỗi sự kiện có hai mốc thời gian khác nhau:

- **Event time**: lúc sự việc thật sự xảy ra, ví dụ lúc một người bấm lưu chỉnh sửa trên Wikipedia.
- **Processing time**: lúc hệ thống nhận được sự kiện đó.

Hai mốc này lệch nhau vì mạng chậm, nguồn mất kết nối rồi gửi bù, hay hệ thống tạm dừng. Hình 2.1 cho thấy ba hiện tượng mà mọi hệ thống luồng phải xử lý.



Hình 2.1: Event time và processing time. Mỗi mũi tên nối lúc sự kiện xảy ra với lúc hệ thống nhận được. Sự kiện có thể đến muộn và lộn thứ tự (e3), đến hai lần khi nguồn gửi lại (e5), hoặc đến ở dạng hỏng không đọc được.

Nếu đếm theo processing time, kết quả phụ thuộc vào việc mạng nhanh hay chậm, và chạy lại cùng dữ liệu sẽ ra số khác. Vì vậy WikiPulse đếm theo **event time**: chạy lại bao nhiêu lần cũng ra cùng một kết quả. Cái giá phải trả là hệ thống phải chờ sự kiện muộn, phải tự loại bản trùng và tách riêng bản ghi hỏng. Các mục sau giải thích cách làm từng việc.

2.2 Luồng sự kiện thay đổi của Wikipedia

Wikimedia công bố mọi thay đổi trên các dự án của mình qua dịch vụ EventStreams [8]. Client mở một kết nối HTTP dài và nhận từng sự kiện dạng JSON ngay khi nó xảy ra (giao thức Server-Sent Events). Bảng 2.1 liệt kê các thông tin của một sự kiện mà hệ thống dùng tới.

Bảng 2.1: Thông tin chính trong một sự kiện thay đổi và cách hệ thống dùng

Thông tin	**Ý nghĩa**	**Dùng để**
Định danh sự kiện	Duy nhất cho mỗi sự kiện	Loại bản trùng
Thời điểm	Lúc thay đổi xảy ra (event time)	Xếp vào cửa sổ thời gian
Wiki, trang	Ví dụ “enwiki”, tên bài viết	Nhóm kết quả theo wiki, theo trang
Loại thay đổi	Sửa trang, tạo trang mới, ghi nhật ký	Đếm từng loại
Kích thước trước, sau	Số byte của trang	Phát hiện xóa nhiều nội dung
Người sửa, cờ ẩn danh, cờ bot	Tài khoản hoặc địa chỉ IP	Đếm sửa ẩn danh, sửa bởi bot; được băm trước khi lưu

Kết nối dài có thể đứt. Khi nối lại, client có thể nhận lại một phần sự kiện đã nhận, nên phía sau phải chịu được bản trùng. Ngoài sự kiện thật, Wikimedia còn phát định kỳ

sự kiện canary để tự kiểm tra hệ thống của họ; các sự kiện này phải bị loại khỏi số liệu.

Ghi chú: Bổ sung cách nối lại không mất sự kiện khi làm collector LIVE (W2-THANH-01).

2.3 Apache Kafka: bộ đệm sự kiện bền vững

Kafka [4] lưu sự kiện thành một **nhật ký chỉ ghi thêm** (append-only log). Các khái niệm chính (Hình 2.2):

- **Topic**: một kênh sự kiện, ví dụ kênh chứa các thay đổi thô của Wikipedia.
- **Partition**: topic được chia thành nhiều nhật ký song song để chia tải. Mỗi sự kiện trong một partition có số thứ tự tăng dần gọi là **offset**.
- **Khóa**: producer gắn khóa cho mỗi sự kiện; sự kiện cùng khóa luôn vào cùng partition, nên giữ đúng thứ tự với nhau. WikiPulse dùng khóa là wiki và trang, để các chỉnh sửa của cùng một trang không bị đảo thứ tự.
- **Consumer** tự nhớ mình đã đọc đến offset nào, nên có thể dừng rồi đọc tiếp, hoặc đọc lại từ đầu.



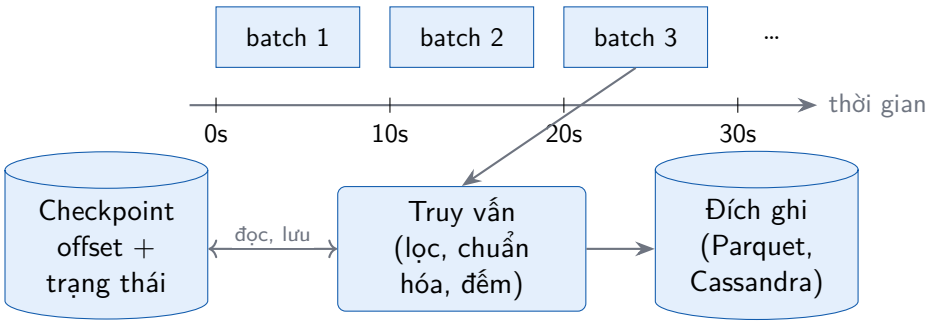
Hình 2.2: Topic Kafka gồm ba partition. Mỗi ô là một sự kiện, số trong ô là offset. Producer ghi vào cuối nhật ký; consumer (Spark) đọc theo offset và nhớ vị trí đã đọc. Nét đứt là phần chưa làm.

Trong WikiPulse, Kafka đứng giữa nguồn sự kiện và Spark. Spark dừng để nâng cấp hay bị lỗi thì sự kiện vẫn nằm an toàn trong Kafka; khi Spark chạy lại, nó đọc tiếp từ offset cũ. Nhờ đọc lại được, chế độ REPLAY chỉ cần ghi dữ liệu mẫu vào cùng topic là toàn bộ phần sau chạy y như với dữ liệu thật.

2.4 Spark Structured Streaming

2.4.1 Mô hình micro-batch

Spark Structured Streaming [2] cho phép viết truy vấn trên luồng giống như truy vấn trên một bảng bình thường. Luồng được coi là một **bảng không giới hạn**, mỗi sự kiện mới là một dòng thêm vào. Cứ sau một khoảng cố định (10 giây trong WikiPulse), Spark lấy phần dữ liệu mới thành một **micro-batch**, tính tiếp kết quả từ trạng thái lần trước, rồi ghi kết quả ra đích (Hình 2.3).



Hình 2.3: Mô hình micro-batch. Mỗi 10 giây, phần dữ liệu mới thành một batch; truy vấn kết hợp batch với trạng thái lưu trong checkpoint để ra kết quả mới.

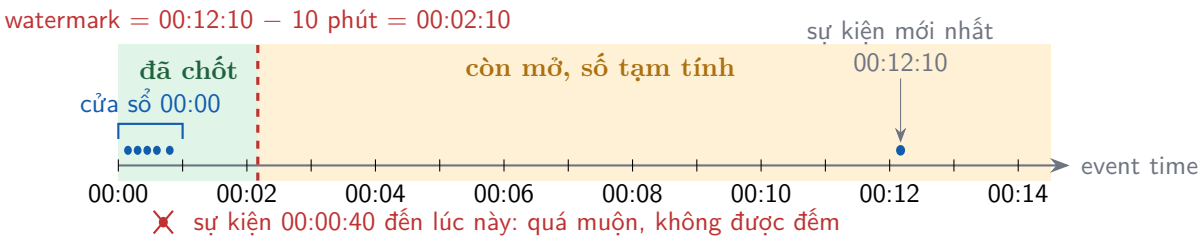
Checkpoint lưu hai thứ: đã đọc Kafka tới offset nào, và trạng thái đang tính dở (ví dụ số đếm của các cửa sổ chưa chốt). Khi Spark khởi động lại, nó đọc checkpoint và chạy tiếp như chưa từng dừng. Checkpoint gắn chặt với cấu trúc truy vấn: đổi cách tính có trạng thái thì phải dùng checkpoint mới.

2.4.2 Cửa sổ thời gian và watermark

Để đếm "mỗi phút có bao nhiêu chỉnh sửa", sự kiện được xếp vào các **cửa sổ** 1 phút theo event time. Câu hỏi khó là: *khi nào thì một cửa sổ được coi là đủ?* Sự kiện của phút 00:00 có thể vẫn đang trên đường tới. Spark trả lời bằng **watermark** [1]:

$$\text{watermark} = \text{event time lớn nhất đã thấy} - \text{độ trễ cho phép (10 phút)}.$$

Cửa sổ nào kết thúc trước watermark thì được **chốt**: kết quả không đổi nữa và trạng thái của nó được xóa khỏi bộ nhớ. Sự kiện đến muộn hơn watermark bị bỏ khỏi phép đếm (nhưng vẫn được lưu ở tầng thô để tra lại). Hình 2.4 minh họa bằng chính dữ liệu mẫu của dự án.



Hình 2.4: Watermark trên dữ liệu mẫu. Sự kiện lúc 00:12:10 đẩy watermark tới 00:02:10, nên cửa sổ 00:00 (5 chỉnh sửa enwiki) được chốt. Mọi cửa sổ sau watermark vẫn mở và có thể còn thay đổi.

Độ trễ cho phép là một sự đánh đổi: dài thì đếm được nhiều sự kiện muộn hơn nhưng phải giữ nhiều trạng thái và chờ lâu mới có số chốt; ngắn thì ngược lại. Dự án chọn 10 phút.

2.4.3 Số tạm tính và số đã chốt

Người xem dashboard muốn thấy số ngay, không muốn chờ 10 phút. Spark có hai **chế độ xuất** kết quả, và WikiPulse dùng cả hai cho cùng một phép đếm (Bảng 2.2).

Bảng 2.2: Hai chế độ xuất kết quả

Chế độ	Khi nào ghi	Tính chất	Đích trong WikiPulse
Update	Mỗi batch, mọi cửa sổ vừa thay đổi	Nhanh; số còn có thể tăng (tạm tính)	Cassandra, cho API
Append	Chỉ khi cửa sổ đã qua watermark	Chậm hơn ít nhất 10 phút; số không đổi nữa (đã chốt)	Tầng Gold, cho phân tích

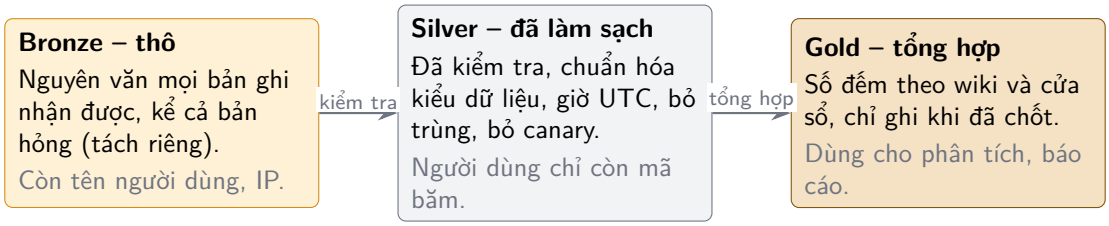
Spark có một giới hạn quan trọng: một truy vấn ở chế độ Update không được chứa hai bước cần nhớ trạng thái liên tiếp, ví dụ "loại trùng rồi đếm" [7]. Vì vậy WikiPulse tách thành nhiều truy vấn: một truy vấn loại trùng và ghi ra đĩa, các truy vấn khác đọc lại kết quả đó để đếm (Mục 4.7).

2.4.4 Loại bản trùng

Mỗi sự kiện có một định danh duy nhất. Spark nhớ các định danh đã thấy và bỏ sự kiện nào trùng. Không thể nhớ mãi, nên Spark chỉ nhớ trong phạm vi watermark: bản trùng đến cách bản gốc quá 10 phút thì không còn bị phát hiện. Đây là giới hạn có chủ đích, được ghi rõ trong contract của nhóm.

2.5 Kiến trúc dữ liệu ba tầng

Thay vì chỉ lưu kết quả cuối, dữ liệu được lưu theo ba tầng [3], mỗi tầng sạch hơn tầng trước (Hình 2.5). Lưu tầng thô giúp kiểm tra lại khi nghi có lỗi và tính lại khi đổi công thức mà không cần nguồn gửi lại dữ liệu.



Hình 2.5: Ba tầng dữ liệu. Đi từ trái sang phải, dữ liệu sạch dần và ít thông tin nhạy cảm hơn.

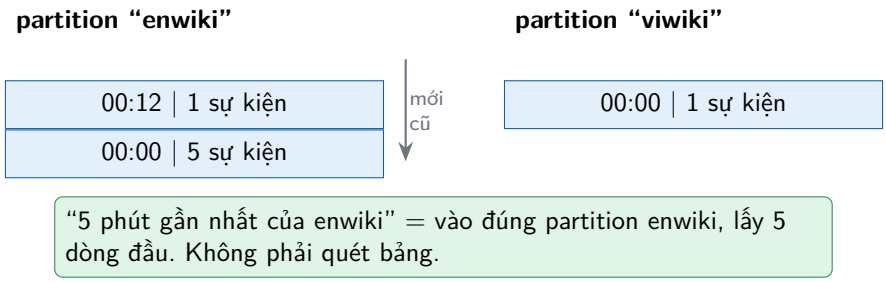
Ba tầng được lưu dạng **Parquet**, định dạng theo cột, nén tốt, đọc nhanh khi chỉ cần vài cột. File được chia thư mục theo ngày và giờ, nên truy vấn một khoảng thời gian chỉ phải đọc đúng các thư mục đó.

2.6 Apache Cassandra: phục vụ kết quả

Dashboard cần đọc số liệu nhanh, liên tục. Parquet hợp cho phân tích theo lô nhưng không hợp để trả lời hàng nghìn câu hỏi nhỏ mỗi phút. Kết quả tạm tính vì vậy được ghi vào Cassandra [5].

Khác CSDL quan hệ, trong Cassandra **bảng được thiết kế theo câu truy vấn**, không theo thực thể [6]. Khóa chính có hai phần (Hình 2.6):

- **Partition key** quyết định dữ liệu nằm cùng nhau ở đâu. WikiPulse dùng wiki: mọi cửa sổ của enwiki nằm chung một partition.
- **Clustering key** quyết định thứ tự trong partition. WikiPulse dùng thời điểm bắt đầu cửa sổ, sắp giảm dần, nên các phút mới nhất luôn nằm đầu.



Hình 2.6: Dữ liệu hoạt động trong Cassandra. Mỗi wiki là một partition; trong partition, các cửa sổ xếp mới trước.

Hai tính chất của Cassandra quyết định cách ghi:

- Ghi một dòng có khóa đã tồn tại thì **ghi đè** (upsert), không sinh dòng mới. WikiPulse ghi *tổng số* của cửa sổ chứ không ghi “cộng thêm 1”, nên khi Spark ghi lại một batch sau sự cố, kết quả vẫn đúng, không bị đếm đôi.
- Truy vấn không chỉ rõ partition buộc Cassandra quét mọi node. Dự án cấm kiểu truy vấn này (“allow filtering”); mỗi câu hỏi của API phải có bảng thiết kế riêng cho nó.

2.7 FastAPI và Streamlit

FastAPI là framework Python để viết API web; kiểu dữ liệu khai báo một lần được dùng để kiểm tra đầu vào, định dạng đầu ra và sinh tài liệu API tự động (OpenAPI). **Streamlit** dựng giao diện dashboard bằng Python thuần, hợp với nhóm không có người chuyên frontend.

2.8 Công nghệ sử dụng

Bảng 2.3: Công nghệ sử dụng và vai trò

Thành phần	**Lựa chọn**	**Vai trò**
Hàng đợi	Apache Kafka 3.9	Giữ sự kiện, tách nguồn khỏi xử lý, cho phép đọc lại
Xử lý luồng	Spark 3.5 (PySpark)	Kiểm tra, chuẩn hóa, loại trùng, đếm theo cửa sổ
Lưu trữ phân tích	Parquet (cục bộ, sau là HDFS)	Ba tầng Bronze, Silver, Gold
Lưu trữ phục vụ	Apache Cassandra 4.1	Số tạm tính cho API
API	FastAPI	Cung cấp số liệu cho dashboard
Giao diện	Streamlit	Dashboard bốn trang
Chạy	Docker Compose	Dựng toàn bộ hệ thống bằng một lệnh, chạy trên ARM64 và x86_64
Ngôn ngữ, kiểm thử	Python 3.11, pytest	Toàn bộ mã nguồn và kiểm thử

Ghi chú: Thêm HDFS khi có profile đầy đủ (tuần 2–3).

Chương 3

Các công cụ và nghiên cứu liên quan

3.1 Công cụ tuần tra của cộng đồng Wikipedia

(Dự kiến, chưa thực hiện.)

3.2 Mô hình đánh giá chỉnh sửa của Wikimedia

(Dự kiến, chưa thực hiện.)

3.3 Thống kê lượt xem và xu hướng trang

(Dự kiến, chưa thực hiện.)

3.4 Khoảng trống mà WikiPulse nhắm tới

(Dự kiến, chưa thực hiện.)

Chương 4

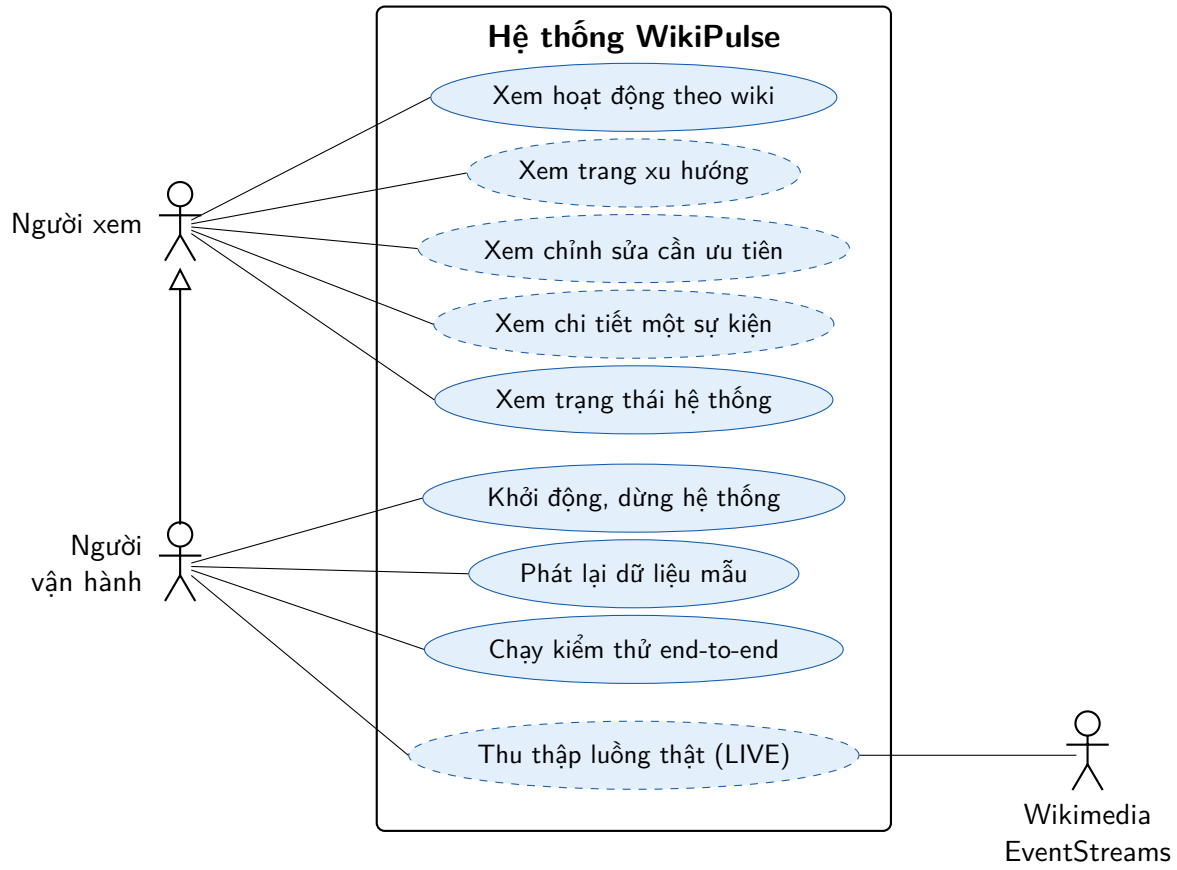
Phân tích, thiết kế và cài đặt

4.1 Phân tích yêu cầu

4.1.1 Tác nhân

- **Người xem:** người tuần tra hoặc người quan tâm đến Wikipedia, mở dashboard để xem hoạt động, trang xu hướng và các chỉnh sửa nên xem trước. Không cần đăng nhập (ngoài phạm vi MVP).
- **Người vận hành:** thành viên nhóm khởi động, dừng hệ thống, phát lại dữ liệu mẫu, chuyển sang nguồn thật và kiểm tra trạng thái. Người vận hành làm được mọi việc của người xem.
- **Wikimedia EventStreams (hệ thống ngoài):** phát luồng sự kiện thay đổi ở chế độ LIVE.

4.1.2 Biểu đồ use case



Hình 4.1: Biểu đồ use case tổng quát. Đường nối là quan hệ giữa tác nhân và use case; mũi tên tam giác rỗng là quan hệ khái quát hóa (người vận hành làm được mọi việc của người xem). Use case viền đứt nét là phần chưa làm.

Ghi chú: Viền đứt nét đổi thành liền khi use case tương ứng xong (tuần 2–3).

4.1.3 Đặc tả các use case chính

Bảng 4.1: Đặc tả use case "Xem hoạt động theo wiki"

Tác nhân	Người xem
Tiền điều kiện	Hệ thống đang chạy
Luồng chính	1. Người xem mở trang Overview, chọn wiki (mặc định enwiki). 2. Dashboard gọi API lấy các cửa sổ 1 phút mới nhất của wiki đó. 3. API đọc đúng partition của wiki trong Cassandra và trả kết quả, gắn nhãn "tạm tính". 4. Dashboard hiện năm chỉ số (sự kiện, chỉnh sửa, trang mới, sửa ẩn danh, sửa bởi bot) và biểu đồ theo thời gian; tự làm mới sau 15 giây. 5. Người xem có thể chọn một cửa sổ cụ thể để xem riêng.
Luồng thay thế	3a. Chưa có dữ liệu: hiện thông báo trống, không hiện số giả. 3b. Cassandra không phản hồi: API trả lỗi "phụ thuộc chưa sẵn sàng"; dashboard báo dịch vụ chưa sẵn sàng. 1a. Wiki không nằm trong danh sách cho phép: API từ chối yêu cầu.
Hậu điều kiện	Không thay đổi dữ liệu

Bảng 4.2: Đặc tả use case "Phát lại dữ liệu mẫu"

Tác nhân	Người vận hành
Tiền điều kiện	Kafka đã sẵn sàng và đã có topic
Luồng chính	1. Người vận hành chạy lệnh phát lại. 2. Hệ thống đọc từng dòng của file dữ liệu mẫu, giữ nguyên định danh và thời điểm của sự kiện, thêm thời điểm nhận. 3. Hệ thống gửi từng sự kiện vào Kafka với khóa wiki và trang, chờ Kafka xác nhận. 4. Spark xử lý như với dữ liệu thật; sau khoảng 10–30 giây số liệu xuất hiện trên dashboard.
Luồng thay thế	2a. Dòng không phải JSON hợp lệ: vẫn gửi nguyên văn để kiểm tra đường xử lý bản ghi hỏng. 1a. Người vận hành chọn tốc độ phát nhanh hơn: chỉ đổi nhịp gửi, không đổi thời điểm sự kiện.
Hậu điều kiện	Kafka có thêm các sự kiện mẫu; phát lại nhiều lần không làm tăng số đếm trong phạm vi watermark

Đặc tả use case "Xem trang xu hướng"

(Dự kiến, chưa thực hiện.)

Đặc tả use case "Xem chỉnh sửa cần ưu tiên"

(Dự kiến, chưa thực hiện.)

Đặc tả use case "Xem chi tiết một sự kiện"

(Dự kiến, chưa thực hiện.)

Đặc tả use case "Thu thập luồng thật (LIVE)"

(Dự kiến, chưa thực hiện.)

4.1.4 Yêu cầu chức năng

Bảng 4.3: Yêu cầu chức năng và trạng thái

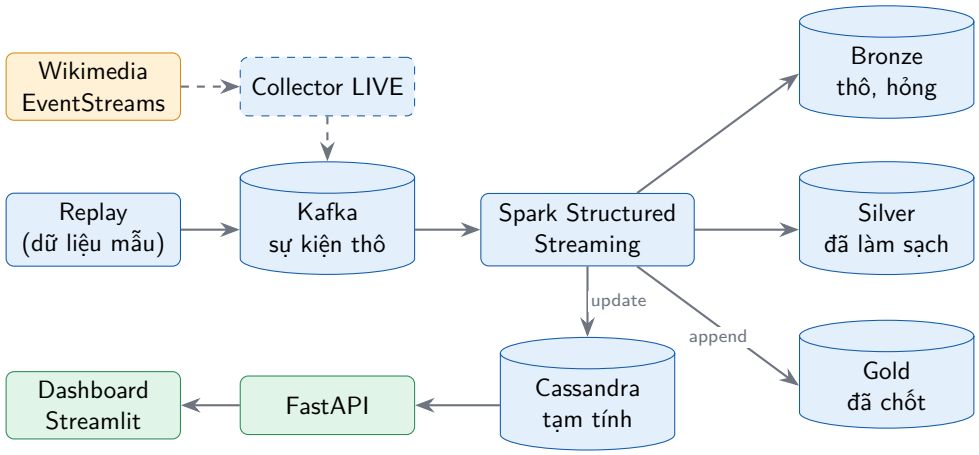
Mã	Yêu cầu	Trạng thái
F1	Phát lại dữ liệu mẫu vào Kafka, giữ nguyên định danh và thời điểm sự kiện	Đã làm
F2	Nhận luồng thật từ Wikimedia, tự nối lại khi mất kết nối	Tuần 2
F3	Kiểm tra, tách riêng bản ghi hỏng, chuẩn hóa, loại trùng	Đã làm
F4	Đếm hoạt động theo wiki, cửa sổ 1 phút; có số tạm tính và số đã chốt	Đã làm
F5	Trang xu hướng: cửa sổ 5 phút trượt mỗi 1 phút, top-N theo wiki	Tuần 2
F6	Ưu tiên xem xét chỉnh sửa theo ba dấu hiệu: xóa nhiều nội dung, hoàn tác qua lại, sửa ẩn danh	Tuần 2
F7	Tra cứu chi tiết một sự kiện	Tuần 2–3
F8	Dashboard 4 trang: Overview, Trending, Suspicious Edits, System Status	Overview đã làm
F9	Lưu ba tầng trên HDFS	Tuần 2–3

Ghi chú: Cập nhật trạng thái theo từng checkpoint.

4.1.5 Yêu cầu phi chức năng

- **Độ trễ:** số tạm tính hiện trên API trong 30–60 giây ở tải demo (mục tiêu, chưa đo). Số đã chốt chậm hơn theo watermark 10 phút.
- **Đúng khi chạy lại:** retry hay khởi động lại không được làm số bị đếm đôi.
- **Riêng tư:** ngoài tầng thô, không nơi nào chứa tên người dùng hay IP. Người dùng được thay bằng mã băm có "muối"; đây là bút danh hóa, không phải ẩn danh hóa hoàn toàn.
- **Tái hiện được:** mọi kết quả kiểm thử chạy lại được bằng lệnh có sẵn.
- **Tài nguyên:** chạy được trên máy 16 GB RAM, cấp cho Docker 6–8 GB.

4.2 Kiến trúc tổng quan



Hình 4.2: Sơ đồ kiến trúc tổng quan. Nét đứt là phần chưa làm (collector LIVE, tuần 2). Màu: xanh dương là thành phần xử lý, xanh lá là đầu ra cho người dùng, vàng là nguồn bên ngoài.

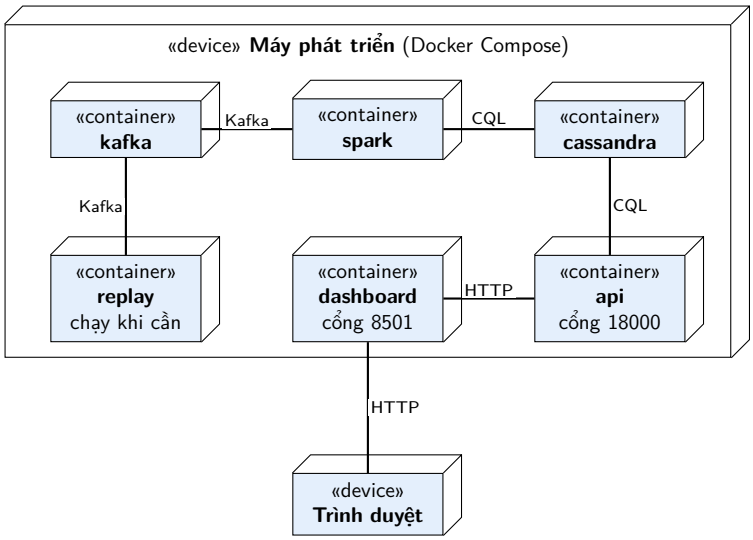
Kafka tách phần nhận khỏi phần xử lý; Spark là nơi duy nhất biến đổi dữ liệu; API và dashboard chỉ đọc kết quả. Dashboard không đọc Cassandra trực tiếp mà luôn đi qua API, để API là điểm duy nhất áp quy tắc riêng tư và giới hạn truy vấn.

4.2.1 Các thành phần và người phụ trách

Bảng 4.4: Các thành phần (vị trí mã nguồn ở Phụ lục A)

Thành phần	**Nhiệm vụ**	**Phụ trách**
Lõi dùng chung	Đọc và kiểm tra cấu hình, ghi log dạng JSON, lớp truy cập Cassandra	Trọng, Nhân
Thu thập	Phát lại dữ liệu mẫu, thu thập luồng thật	Thành
Xử lý	Tiến trình Spark: các truy vấn, phép biến đổi, giám sát truy vấn	Linh
Ghi kết quả	Đưa số đếm từ Spark vào Cassandra	Nhân
Phục vụ	Lược đồ Cassandra, API	Nhân
Giao diện	Dashboard và lớp gọi API	Mạnh
Tích hợp	Docker Compose, tạo topic, dữ liệu mẫu, kiểm thử end-to-end	Trọng

4.2.2 Triển khai

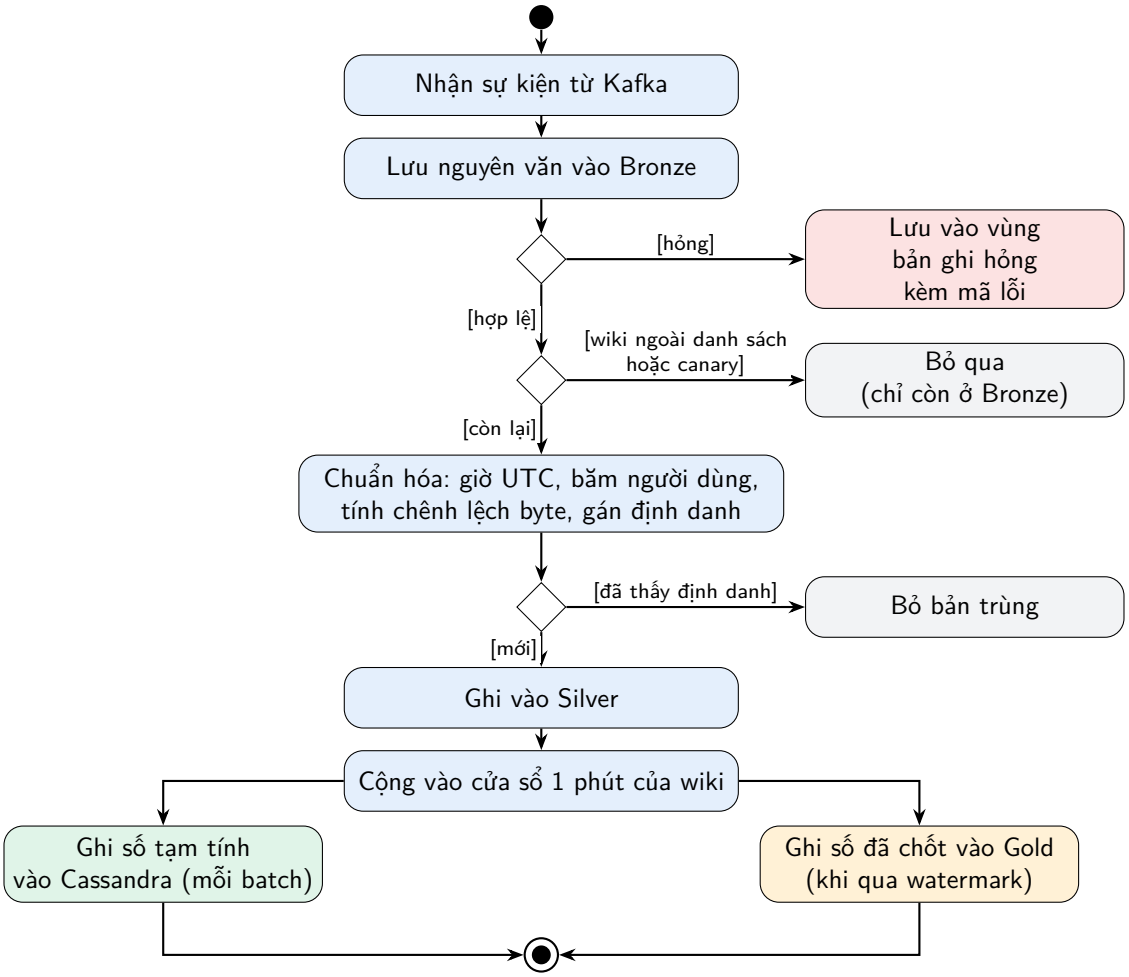


Hình 4.3: Biểu đồ triển khai hiện tại. Các container gọi nhau bằng tên dịch vụ trong mạng Docker; chỉ API, dashboard và Kafka mở cổng ra máy chủ, và chỉ cho chính máy đó truy cập. Hai container khởi tạo (tạo topic, tạo lược đồ Cassandra) chạy một lần trước các dịch vụ chính.

Ghi chú: Thêm NameNode/DataNode khi có profile HDFS (W2-TRONG-01); Kubernetes ở tuần 3.

4.3 Đường đi của một sự kiện

Hình 4.4 cho thấy một sự kiện đi qua những bước kiểm tra nào trước khi được đếm. Mỗi nhánh rẽ đều để lại dấu vết: không có sự kiện nào "biến mất" mà không được lưu ở tầng thô.



Hình 4.4: Biểu đồ hoạt động: đường đi của một sự kiện. Hình thoi là nút rẽ nhánh, điều kiện ghi trong ngoặc vuông. Bốn loại lỗi được ghi mã riêng: không phải JSON hợp lệ, thiếu thời điểm, thiếu wiki, thiếu trang.

4.4 Biểu đồ tuần tự: người xem mở trang Overview

(Dự kiến, chưa thực hiện.)

Ghi chú: Vẽ sau khi W1-MANH-01 xong, lúc luồng dashboard → API → Cassandra đã ổn định.

4.5 Cấu hình và tách lần chạy

Mọi tham số (chế độ LIVE hay REPLAY, danh sách wiki, độ trễ watermark, địa chỉ các dịch vụ…) được đọc từ biến môi trường và kiểm tra ngay khi khởi động, trước khi mở kết nối tốn tài nguyên. Mỗi lần chạy có một **mã lần chạy**; dữ liệu và checkpoint của lần chạy nằm trong thư mục riêng mang mã đó, và mỗi truy vấn Spark có checkpoint riêng. Khi đổi schema hay nguồn dữ liệu, nhóm tạo mã lần chạy và keyspace Cassandra mới chứ

không xóa checkpoint rồi ghi tiếp vào kết quả cũ. Kiểm thử end-to-end dùng mã lần chạy và vùng lưu trữ riêng nên không đụng dữ liệu đang dùng để phát triển.

4.6 Thu thập dữ liệu

4.6.1 Kafka

Hệ thống dùng ba topic: sự kiện thô (3 partition), bản ghi không xử lý được (dead-letter queue) và cảnh báo của đường ống; một broker, mỗi partition một bản sao. Sự kiện giữ nguyên JSON của Wikimedia, chỉ thêm thời điểm nhận và phiên bản bộ thu thập. Khóa là wiki và mã trang (thiếu mã trang thì dùng tên trang).

4.6.2 Phát lại dữ liệu mẫu

Bộ phát lại đọc file dữ liệu mẫu và gửi từng dòng vào Kafka, giữ nguyên định danh và thời điểm sự kiện; tham số tốc độ chỉ đổi nhịp gửi. Dữ liệu mẫu ngày 01/01/2026 có 11 dòng, được chọn để thử đủ các nhánh ở Hình 4.4: 5 chỉnh sửa enwiki, 1 trang mới viwiki, 1 bản trùng, 2 bản hỏng, 1 sự kiện canary, và 1 sự kiện lúc 00:12:10 để đẩy watermark qua cửa sổ 00:00 (Hình 2.4).

4.6.3 Thu thập luồng thật

(Dự kiến, chưa thực hiện.)

4.7 Xử lý luồng

4.7.1 Các truy vấn

Một tiến trình Spark chạy năm truy vấn độc lập, mỗi truy vấn một checkpoint (Bảng 4.5). Bước loại trùng ghi kết quả ra Silver; hai truy vấn đếm đọc lại Silver. Nhờ vậy không truy vấn nào vừa loại trùng vừa đếm ở chế độ Update (giới hạn đã nêu ở Chương 2).

Bảng 4.5: Các truy vấn streaming

Truy vấn	**Việc làm**	**Ghi vào (chế độ)**
Bronze thô	Đọc Kafka, giữ nguyên bản ghi và vị trí trong Kafka	Parquet (append)
Bronze hỏng	Đọc Kafka, lọc bản ghi hỏng, gắn mã lỗi	Parquet (append)
Silver	Đọc Kafka, lọc, chuẩn hóa, loại trùng	Parquet (append)
Hoạt động tạm tính	Đọc Silver, đếm theo wiki và phút	Cassandra (update)
Hoạt động đã chốt	Đọc Silver, đếm như trên	Parquet Gold (append)

4.7.2 Định danh và loại trùng

Định danh của sự kiện lấy từ nguồn. Nếu nguồn không có, hệ thống tự tạo bằng cách băm các trường không đổi của sự kiện (wiki, trang, phiên bản, thời điểm, loại); không bao giờ dùng thời điểm nhận hay xử lý, vì như vậy cùng một sự kiện gửi hai lần sẽ có hai định danh khác nhau. Mọi phép biến đổi dùng hàm có sẵn của Spark, không viết hàm Python riêng, để Spark tối ưu được và không phải chuyển dữ liệu qua lại giữa Java và Python.

4.7.3 Đếm hoạt động

Với mỗi wiki và mỗi phút, hệ thống đếm: tổng sự kiện hợp lệ, số chỉnh sửa, số trang mới, và trong số chỉnh sửa thì bao nhiêu là ẩn danh, bao nhiêu là của bot.

4.7.4 Đặc trưng xu hướng

(Dự kiến, chưa thực hiện.)

4.7.5 Chấm điểm ưu tiên xem xét

(Dự kiến, chưa thực hiện.)

4.8 Lưu trữ

4.8.1 Tầng Parquet

Bronze và Silver chia thư mục theo ngày và giờ của sự kiện; Gold chia theo ngày, giờ và wiki. Danh sách đầy đủ các trường của Silver nằm trong tài liệu contract của nhóm.

Ghi chú: Chuyển sang HDFS ở tuần 2–3 (W2-TRONG-01, W3-LINH-01); thêm kết quả đọc lại sau khởi động lại.

4.8.2 Lược đồ Cassandra

Bảng hoạt động theo cửa sổ (Hình 4.5) phục vụ đúng hai câu hỏi của API: "các phút mới nhất của một wiki" và "một phút cụ thể của một wiki". Cách chọn khóa đã giải thích ở Hình 2.6.

wiki_activity_by_window			
PK	**wiki**	text	khóa partition
CK↓	**window_start**	timestamp	sắp mới trước
	window_end	timestamp	
	event_count	bigint	tổng sự kiện
	edit_count	bigint	chỉnh sửa
	new_page_count	bigint	trang mới
	anonymous_edit_count	bigint	sửa ẩn danh
	bot_edit_count	bigint	sửa bởi bot
	updated_at	timestamp	lần ghi gần nhất

Hình 4.5: Bảng hoạt động theo cửa sổ. PK: khóa partition; CK↓: khóa sắp xếp, giảm dần.

- Spark ghi **tổng số** của cửa sổ, không ghi phần cộng thêm, nên ghi lại sau sự cố không làm đếm đôi.
- Mỗi wiki thêm 1.440 dòng mỗi ngày, khoảng 525 nghìn dòng mỗi năm: đủ cho demo. Trước khi chạy LIVE dài ngày cần đặt thời hạn tự xóa (TTL) hoặc chia partition theo ngày.
- Trước khi ghi, mỗi dòng được kiểm tra: tên wiki hợp lệ, cửa sổ kết thúc sau khi bắt đầu, số đếm nhất quán (chỉnh sửa cộng trang mới không vượt tổng; ẩn danh, bot không vượt số chỉnh sửa). Dòng sai làm batch thất bại để Spark chạy lại, thay vì ghi số sai.

Ghi chú: Thêm bảng xu hướng, chỉnh sửa đáng ngờ, chi tiết sự kiện ở tuần 2 (W2-NHAN-01); TTL ở tuần 3 (W3-NHAN-01).

4.8.3 Ghi kết quả từ Spark sang Cassandra

Ở mỗi batch, tiến trình chính của Spark duyệt lần lượt từng dòng kết quả (không nạp cả batch vào bộ nhớ) và ghi vào Cassandra. Nếu ghi lỗi, truy vấn dừng và Spark chạy lại

batch đó từ checkpoint. Parquet và Cassandra không có giao dịch chung, nên hệ thống không cam kết "đúng một lần" trên cả hai nơi; tính đúng dựa vào việc ghi đè theo khóa ổn định.

4.9 API

Bảng 4.6: Các endpoint

Endpoint	Mô tả	Trạng thái
`/health`	Kiểm tra Cassandra thật sự trả lời được	Đã làm
`/api/v1/activity`	Hoạt động theo cửa sổ; chọn wiki, một cửa sổ cụ thể, số dòng tối đa 100	Đã làm
`/api/v1/trends`	Top-N trang xu hướng	Tuần 2
`/api/v1/suspicious-edits`	Chỉnh sửa cần ưu tiên xem xét	Tuần 2
`/api/v1/events/{id}`	Chi tiết một sự kiện	Tuần 2–3

Mọi phản hồi dữ liệu kèm thời điểm tạo, nguồn, chế độ chạy và nhãn "tạm tính"; mọi thời điểm theo UTC. Không có dữ liệu thì trả danh sách rỗng, không phải lỗi. Tham số sai (wiki ngoài danh sách, cửa sổ thiếu múi giờ, số dòng ngoài 1–100) bị từ chối. Cassandra không trả lời thì API báo "phụ thuộc chưa sẵn sàng"; API vẫn khởi động được khi Cassandra chưa lên và tự hồi phục khi Cassandra sẵn sàng. Endpoint chưa làm trả mã "chưa triển khai" kèm người phụ trách và tuần, để dashboard hiện đúng trạng thái thay vì dữ liệu giả.

Ghi chú: Thống nhất định dạng lỗi cho mọi endpoint và thêm ba endpoint ở tuần 2.

4.10 Dashboard

Bốn trang: Overview (đã đọc dữ liệu thật: chọn wiki và cửa sổ, tự làm mới sau 15 giây), Trending, Suspicious Edits, System Status. Các trang chưa có dữ liệu hiện rõ là chưa triển khai, kèm người phụ trách.

Ghi chú: Mạnh bổ sung ảnh chụp các trang (W1-MANH-01, W2-MANH-01).

Chương 5

Kiểm thử và đánh giá

5.1 Chiến lược kiểm thử

Bảng 5.1: Các tầng kiểm thử

Tầng	**Nội dung**	**Khi chạy**
Nhẹ	Cấu hình, API, lớp truy cập Cassandra, phát lại, dashboard; Cassandra được giả lập ở tầng driver nhưng giữ đúng cách ghi đè theo khóa	Mỗi lần sửa code, vài giây
Spark	Các phép biến đổi và cách điều phối truy vấn, chạy trong container có Java	Khi sửa phần xử lý
Tích hợp	Lớp truy cập trên Cassandra thật, keyspace ngẫu nhiên, tự xóa sau khi chạy	Khi sửa lược đồ, truy vấn
End-to-end	Dựng hệ thống riêng, phát dữ liệu mẫu hai pha, kiểm tra cả ba tầng, API và giao diện	Trước khi nghiệm thu

Lệnh chạy từng tầng ở Phụ lục A.

5.2 Kết quả CP0

Dữ liệu mẫu ngày 01/01/2026 gồm 11 bản ghi. Kết quả chạy thật (Bảng 5.2) khớp bộ kết quả kỳ vọng; bộ này được viết tay độc lập, không sinh ra từ chính code xử lý.

Bảng 5.2: Kết quả CP0 trên dữ liệu mẫu, cửa sổ 2026-01-01 00:00 UTC

Chỉ tiêu	**Kết quả**
Bản ghi Kafka / Bronze raw	11 / 11
Bronze invalid	2
Silver (sau loại trùng và canary)	7
Gold cửa sổ 00:00 (đã chốt)	2 dòng
enwiki: event / edit / trang mới / ẩn danh / bot	5 / 5 / 0 / 1 / 1
viwiki: event / edit / trang mới / ẩn danh / bot	1 / 0 / 1 / 0 / 0

Bảng 5.3: Các lần kiểm chứng

Ngày	**Máy**	**Kết quả**
11/10/2026	Apple M3, ARM64	84 test nhẹ đạt; 16 test Spark đạt; kiểm thử end-to-end đạt (Trọng)
15/10/2026	Windows 11, x86_64	CP0 chạy lại, số khớp Bảng 5.2; phát lại lần hai không đổi số; 33 test API đạt, gồm 4 test trên Cassandra thật (Nhân)

Ghi chú: Mỗi thành viên chạy lại CP0 trên máy mình thì thêm một dòng (CP1).

Bài học khi chạy trên Windows

Hai lỗi chỉ xuất hiện trên Windows. (1) Git trên Windows mặc định đổi ký tự xuống dòng của file sang kiểu Windows (CRLF); script tạo topic chạy trong container Linux vì thế đọc sai và dừng. Cách sửa: khai báo cho Git luôn giữ kiểu Linux (LF) với mọi script shell. (2) Thư viện Cassandra cho Python cần một module đã bị gỡ khỏi Python 3.12; container dùng Python 3.11 nên không bị, còn máy phát triển dùng 3.12 phải cài thêm gói thay thế. Bài học: mỗi thành viên nên chạy lại toàn bộ hệ thống trên máy mình sớm (mốc CP1), vì lỗi môi trường không lộ ra khi chỉ chạy trên một máy.

5.3 Xu hướng và ưu tiên xem xét trên fixture vàng

(Dự kiến, chưa thực hiện.)

5.4 Độ trễ end-to-end

(Dự kiến, chưa thực hiện.)

5.5 Khởi động lại, retry và dữ liệu đến muộn

(Dự kiến, chưa thực hiện.)

5.6 Chạy LIVE liên tục

(Dự kiến, chưa thực hiện.)

5.7 Tải và dung lượng

(Dự kiến, chưa thực hiện.)

Chương 6

Kết luận và hướng phát triển

6.1 Kết quả đạt được

Đến ngày 15/10/2026, khung end-to-end đã chạy thật ở chế độ REPLAY: Kafka nhận sự kiện, Spark kiểm tra, loại trùng, tổng hợp theo event time và ghi ba tầng Parquet, Cassandra phục vụ số tạm tính qua API, dashboard hiển thị trang Overview. Kết quả khớp bộ kỳ vọng độc lập trên hai máy khác kiến trúc. Các nguyên tắc đúng khi retry (upsert tổng tuyệt đối), riêng tư (không có định danh ngoài Bronze) và không dùng dữ liệu giả đã có test bảo vệ.

Ghi chú: Viết lại khi chốt CP4.

6.2 Hạn chế

- Một node, chưa có xác thực, phân quyền hay độ sẵn sàng cao.
- Không có giao dịch chung giữa các sink; tính đúng dựa vào upsert theo khóa ổn định.
- Bronze giữ payload gốc có thể chứa tên người dùng, IP; cần chính sách lưu giữ.

6.3 Hướng phát triển

Các mục dưới đây đã có trong kế hoạch nhưng chưa thực hiện; mỗi mục được viết đầy đủ khi hoàn thành.

6.3.1 Collector LIVE với nối lại và đếm lỗi (W2-THANH-01)

(Dự kiến, chưa thực hiện.)

6.3.2 Cửa sổ trượt, đặc trưng xu hướng và điểm rủi ro (W2-LINH-01)

(Dự kiến, chưa thực hiện.)

6.3.3 API xu hướng, chỉnh sửa đáng ngờ và chi tiết sự kiện (W2-NHAN-01)

(Dự kiến, chưa thực hiện.)

6.3.4 Hoàn thiện bốn trang dashboard (W2-MANH-01)

(Dự kiến, chưa thực hiện.)

6.3.5 Profile HDFS và tích hợp ba tầng (W2-TRONG-01, W3-LINH-01)

(Dự kiến, chưa thực hiện.)

6.3.6 Retry, latest window, TTL và kiểm thử dung lượng (W3-NHAN-01)

(Dự kiến, chưa thực hiện.)

6.3.7 Khung Kubernetes và monitoring (W3-TRONG-01)

(Dự kiến, chưa thực hiện.)

6.3.8 Đóng băng, clone sạch và demo (tuần 4)

(Dự kiến, chưa thực hiện.)

Phụ lục A

Bảng tra mã nguồn

Phụ lục này nối các khái niệm trong báo cáo với vị trí mã nguồn, để người đọc tìm code mà không phải đọc tên file trong phần nội dung. Đường dẫn tính từ thư mục gốc của repo.

Bảng A.1: Khái niệm và vị trí mã nguồn

Khái niệm	**Vị trí**
Cấu hình, kiểm tra tham số khởi động	`shared/wikipulse_core/config.py`
Lớp truy cập Cassandra, kiểm tra dòng trước khi ghi	`shared/wikipulse_core/serving.py`
Tạo topic Kafka	`scripts/create_topics.sh`
Phát lại dữ liệu mẫu	`replay/replay_producer.py`
Dữ liệu mẫu và kết quả kỳ vọng	`tests/fixtures/`
Tiến trình Spark và năm truy vấn	`processing/spark/jobs/streaming_job.py`
Kiểm tra, chuẩn hóa, loại trùng, đếm	`processing/spark/wikipulse/transformations.py`
Ghi kết quả sang Cassandra	`processing/spark/wikipulse/sinks.py`
Thư mục theo mã lần chạy	`processing/spark/wikipulse/paths.py`
Lược đồ Cassandra	`storage/cassandra/schema.cql`
API	`api/app/main.py`, `api/app/models.py`
Dashboard	`dashboard/app.py`, `dashboard/views/`
Contract giữa các module	`docs/contracts.md`
Bảng việc và trạng thái	`docs/checkpoints.md`

Bảng A.2: Lệnh thường dùng (chạy ở thư mục gốc repo)

Lệnh	**Tác dụng**
`make install`	Tạo môi trường Python và cài thư viện
`make up`	Dựng và khởi động toàn bộ hệ thống
`make replay`	Phát lại dữ liệu mẫu vào Kafka
`make down`	Dừng hệ thống, giữ dữ liệu
`make test`	Chạy kiểm thử nhẹ
`make test-spark`	Chạy kiểm thử Spark trong container
`pytest -m integration api/tests`	Kiểm thử trên Cassandra thật (cần đặt địa chỉ Cassandra qua biến môi trường)
`make smoke`	Kiểm thử end-to-end trên hệ thống riêng, tự dọn sau khi chạy

Tài liệu tham khảo

[1] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, and Sam Whittle. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment*, 8(12):1792–1803, 2015.

[2] Michael Armbrust, Tathagata Das, Joseph Torres, Burak Yavuz, Shixiong Zhu, Reynold Xin, Ali Ghodsi, Ion Stoica, and Matei Zaharia. Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pages 601–613, 2018.

[3] Databricks. What is the medallion lakehouse architecture? `https://docs.databricks.com/aws/en/lakehouse/medallion`. Truy cập 15/10/2026.

[4] Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: a Distributed Messaging System for Log Processing. In *Proceedings of the NetDB Workshop*, 2011.

[5] Avinash Lakshman and Prashant Malik. Cassandra: A Decentralized Structured Storage System. *ACM SIGOPS Operating Systems Review*, 44(2):35–40, 2010.

[6] The Apache Software Foundation. Apache Cassandra Documentation: Data Manipulation (CQL). `https://cassandra.apache.org/doc/latest/cassandra/developing/cql/dml.html`. Truy cập 15/10/2026.

[7] The Apache Software Foundation. Structured Streaming Programming Guide, Spark 3.5.6. `https://spark.apache.org/docs/3.5.6/structured-streaming-programming-guide.html`. Truy cập 15/10/2026.

[8] Wikimedia Foundation. EventStreams. https://wikitech.wikimedia.org/wiki/Event_Platform/EventStreams. Truy cập 15/10/2026.